

G2_Clasificador_SenalizacionUrbana_v5_512x512_Final

May 21, 2026

1 Clasificador CNN — Señalización Urbana

1.1 Estado, Visibilidad y Cumplimiento

Tema: La señalización vial y peatonal puede estar deteriorada, ausente, mal ubicada u obstruida, afectando la seguridad vial.

Pregunta de investigación: ¿Puede una CNN clasificar el estado visual de la señalización urbana y apoyar una auditoría visual preliminar?

1.1.1 Mejoras implementadas en esta versión (v5)

Mejora	Descripción
Resolución 512×512	Mayor detalle visual por imagen para mejor extracción de características
80 imágenes por clase	Dataset balanceado y ampliado para las 3 clases
120 épocas	Mayor tiempo de entrenamiento para mejor convergencia
Cross Validation (k=5)	Particionado robusto con validación cruzada estratificada
Matriz de confusión Train	Evaluación adicional sobre el conjunto de entrenamiento
AutoAugment (ImageNet)	Política de augmentation aprendida automáticamente
TrivialAugmentWide	Augmentation simple y fuerte para datasets pequeños
Functional Transforms	Transformaciones manuales para simular condiciones viales reales
CNN más profunda	5 bloques convolucionales + Residual Skip Connection
WeightedRandomSampler	Muestreo balanceado para compensar desbalance entre clases
Gradient Clipping	Estabiliza el entrenamiento evitando gradientes explosivos
Sin RandomHorizontalFlip / RandomVerticalFlip	Eliminados para no introducir orientaciones inválidas en señales

1.1.2 Estructura esperada en Google Drive

```
MyDrive/SenalesTransito/  
  visible_legible/      <- 80 imágenes  
  deteriorada/          <- 80 imágenes  
  poco_visible/         <- 80 imágenes
```

1.2 0. Montar Google Drive

```
[ ]: # Conectar el entorno de Colab con Google Drive  
# Esto permite leer y guardar archivos desde la nube  
from google.colab import drive  
drive.mount('/content/drive')  
print('Google Drive montado correctamente.')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.
Google Drive montado correctamente.

1.3 1. Importaciones y configuración global

```
[ ]: # Librerías estándar y de terceros  
import os                # manejo de rutas y directorios  
import time              # medición de tiempos de entrenamiento  
import numpy as np       # operaciones numéricas  
import matplotlib.pyplot as plt  # visualización de imágenes y métricas  
import seaborn as sns    # matriz de confusión con estilo  
import cv2               # análisis de métricas de imagen (brillo, ↵  
    ↵nitidez)  
from collections import Counter  # conteo de clases para balanceo  
  
# PyTorch: framework principal de la CNN  
import torch  
import torch.nn as nn  
import torch.nn.functional as F  
import torch.optim as optim  
from torch.utils.data import Dataset, DataLoader, random_split, ↵  
    ↵WeightedRandomSampler  
from torchvision import transforms  
from torchvision.transforms import functional as TF  # Functional Transforms  
from torchvision.transforms import autoaugment      # AutoAugment y ↵  
    ↵TrivialAugmentWide  
from PIL import Image  
  
# Métricas de evaluación  
from sklearn.metrics import confusion_matrix, classification_report
```

```
# Verificar GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Dispositivo de computo: {device}')
if device.type == 'cuda':
    print(f' GPU: {torch.cuda.get_device_name(0)}')
```

Dispositivo de computo: cuda
GPU: Tesla T4

```
[ ]: # Constantes globales compartidas por todas las clases

# Ruta base en Google Drive donde están las subcarpetas
# IMPORTANTE: ajusta esta ruta si tu carpeta tiene otro nombre o ubicación
BASE_DIR = '/content/drive/MyDrive/SenalesTransito'

# Las 3 clases del proyecto - deben coincidir exactamente con los nombres de las carpetas
CLASES = [
    'visible_legible', # clase 0: señal clara y legible (80 imgs)
    'deteriorada',     # clase 1: señal rota, oxidada o dañada (80 imgs)
    'poco_visible'     # clase 2: mala iluminación o ángulo (80 imgs)
]

# Número de imágenes por clase (balanceado: 80 por cada una)
IMGS_POR_CLASE = {
    'visible_legible': 80,
    'deteriorada':     80,
    'poco_visible':    80
}

NUM_CLASES = len(CLASES) # 3
IMG_SIZE = 512           # resolución 512x512 px - mayor detalle para mejor clasificación
BATCH_SIZE = 8           # lote pequeño acorde al tamaño del dataset
NUM_EPOCHS = 120         # épocas aumentadas para mejor convergencia con augmentation
K_FOLDS = 5              # número de folds para Cross Validation estratificada
SEED = 42                # semilla para reproducibilidad

# Ruta donde se guardará el mejor modelo entrenado
RUTA_MODELO = os.path.join(BASE_DIR, 'modelo_3clases_v5.pth')

torch.manual_seed(SEED)
print(f'Clases definidas ({NUM_CLASES}): {CLASES}')
print(f'Imágenes por clase: {IMGS_POR_CLASE}')
```

```
print(f'Resolucion: {IMG_SIZE}x{IMG_SIZE}x3 Epochs: {NUM_EPOCHS} K-Folds: {K_FOLDS}')
print(f'Ruta del modelo: {RUTA_MODELO}')
```

Clases definidas (3): ['visible_legible', 'deteriorada', 'poco_visible']
 Imagenes por clase: {'visible_legible': 80, 'deteriorada': 80, 'poco_visible': 80}
 Resolucion: 512x512x3 Epochs: 120 K-Folds: 5
 Ruta del modelo: /content/drive/MyDrive/SenalesTransito/modelo_3clases_v5.pth

1.4 2. Carga y verificación de imágenes por clase

```
[ ]: # Verificar que cada carpeta de clase existe y contar sus imágenes

EXT_VALIDAS = ('.jpg', '.jpeg', '.png', '.webp')

print('Verificando estructura de carpetas en Google Drive:')
print('-' * 55)
todo_ok = True
for i, clase in enumerate(CLASES):
    ruta = os.path.join(BASE_DIR, clase)
    if os.path.exists(ruta):
        n = len([f for f in os.listdir(ruta) if f.lower().
        ↪endswith(EXT_VALIDAS)])
        esperadas = IMGS_POR_CLASE[clase]
        estado = 'OK' if n >= esperadas else f'AVISO: se esperaban {esperadas},
        ↪encontradas {n}'
        print(f' [{i}] {clase:<20} {n:>3} imagenes {estado}')
        if n < 5:
            todo_ok = False
    else:
        print(f' [{i}] {clase:<20} CARPETA NO ENCONTRADA')
        todo_ok = False
print('-' * 55)
print('Listo.' if todo_ok else 'ADVERTENCIA: revisa las rutas o agrega imagenes.
    ↪')
```

Verificando estructura de carpetas en Google Drive:

```
-----
[0] visible_legible      80 imagenes OK
[1] deteriorada         80 imagenes OK
[2] poco_visible        80 imagenes OK
-----
```

Listo.

```
[ ]: # Función para cargar rutas e índices de clase
# Recorre cada subcarpeta y construye una lista de (ruta, indice_clase)
```

```
def cargar_imagenes_por_clase(base_dir, clases, extensiones=EXT_VALIDAS):
    datos = []
    for idx, clase in enumerate(clases):
        carpeta = os.path.join(base_dir, clase)
        if not os.path.exists(carpeta):
            print(f' AVISO: carpeta no encontrada -> {carpeta}')
            continue
        for archivo in sorted(os.listdir(carpeta)):
            if archivo.lower().endswith(extensiones):
                datos.append((os.path.join(carpeta, archivo), idx))
    return datos

datos = cargar_imagenes_por_clase(BASE_DIR, CLASES)

print(f'Total de imagenes cargadas: {len(datos)}')
print()
for idx, clase in enumerate(CLASES):
    n = sum(1 for _, lbl in datos if lbl == idx)
    print(f' Clase {idx} - {clase}: {n} imagenes')
```

Total de imagenes cargadas: 240

Clase 0 - visible_legible: 80 imagenes

Clase 1 - deteriorada: 80 imagenes

Clase 2 - poco_visible: 80 imagenes

```
[ ]: # Visualizar muestra de imágenes ORIGINALES (antes de normalizar)
# Se muestran hasta 5 imágenes por clase para verificar que se cargaron bien

MAX_COLS = 5
fig, axes = plt.subplots(NUM_CLASES, MAX_COLS, figsize=(16, NUM_CLASES * 3))
fig.suptitle('Muestra de imagenes originales por clase (antes de normalizar)',
             fontsize=13, fontweight='bold', y=1.01)

for idx, clase in enumerate(CLASES):
    muestras = [(r, l) for r, l in datos if l == idx]
    for col in range(MAX_COLS):
        ax = axes[idx][col]
        if col < len(muestras):
            img = Image.open(muestras[col][0]).convert('RGB')
            ax.imshow(img)
            if col == 0:
                ax.set_ylabel(f'[{idx}] {clase}', fontsize=8, fontweight='bold')
                ax.set_title(f'{img.size[0]}x{img.size[1]}px', fontsize=7)
        else:
            ax.set_visible(False)
```

```
ax.axis('off')

plt.tight_layout()
plt.show()
```



1.5 3. Preprocesamiento y Data Augmentation avanzado

Se aplican **tres niveles de augmentation** para maximizar la variedad de entrenamiento con un dataset de 80 imágenes por clase:

Nota importante: Se han eliminado `RandomHorizontalFlip` y `RandomVerticalFlip` ya que invertir señales de tránsito horizontal o verticalmente produce imágenes inválidas que no existen en el mundo real, introduciendo ruido en el aprendizaje.

1.5.1 Nivel 1 — AutoAugment (política ImageNet)

Política de transformaciones **aprendida automáticamente** sobre ImageNet. Aplica secuencias de transformaciones optimizadas para clasificación visual. Referencia: `torchvision.transforms.AutoAugment`

1.5.2 Nivel 2 — TrivialAugmentWide

Selecciona **una sola transformación aleatoria** de magnitud amplia en cada imagen. Muy efectivo con datasets de tamaño moderado por su simplicidad y cobertura amplia. Referencia: `torchvision.transforms.TrivialAugmentWide`

1.5.3 Nivel 3 — Functional Transforms personalizados

Transformaciones manuales usando `torchvision.transforms.functional` para simular condiciones viales reales: variación de perspectiva, ajuste de exposición, y degradación de nitidez que imitan fotos tomadas en campo.

Pipeline	Transformaciones aplicadas
Entrenamiento	Resize(512) → AutoAugment(ImageNet) → TrivialAugmentWide → Functional (perspectiva/exposición) → ColorJitter → Rotación → ToTensor → Normalize
Evaluación	Resize(512) → ToTensor → Normalize

```
[ ]: # Clase de augmentation funcional personalizada
# Usa torchvision.transforms.functional para simular condiciones viales reales
# NOTA: RandomHorizontalFlip y RandomVerticalFlip han sido eliminados
# porque invertir señales de tránsito genera imágenes irreales.

import random

class AugmentacionVial:
    """
    Transformaciones funcionales personalizadas que simulan condiciones
    de captura de señales viales en campo:
    - Perspectiva aleatoria: imita fotos tomadas en ángulo
    - Ajuste de exposición: simula variaciones de luz solar
    - Degradación de nitidez: imita fotos desde vehículo en movimiento
    - Posterización: reduce la profundidad de bits (simula cámaras de baja_
    ↪calidad)
    Probabilidad de aplicar cada transformación: configurable.
    """
    def __init__(self, p_perspectiva=0.3, p_exposicion=0.4,
                  p_nitidez=0.3, p_posterizar=0.2):
        self.p_perspectiva = p_perspectiva
        self.p_exposicion = p_exposicion
        self.p_nitidez = p_nitidez
        self.p_posterizar = p_posterizar

    def __call__(self, img):
        """
        Aplica cada transformación con su probabilidad asignada.
        img: imagen PIL RGB
        returns: imagen PIL RGB transformada
        """
        # Perspectiva aleatoria: simula foto desde ángulo lateral o diagonal
        if random.random() < self.p_perspectiva:
            img = TF.perspective(
```

```

        img,
        startpoints=[[0,0],[img.width,0],[img.width,img.height],[0,img.
↪height]],
        endpoints=[
            [random.randint(0, 15), random.randint(0, 15)],
            [img.width - random.randint(0,15), random.randint(0,15)],
            [img.width - random.randint(0,15), img.height - random.
↪randint(0,15)],
            [random.randint(0,15), img.height - random.randint(0,15)]
        ]
    )

    # Ajuste de exposición: brillo + contraste para imitar luz solar
↪variable
    if random.random() < self.p_exposicion:
        factor_brillo = random.uniform(0.6, 1.6)
        factor_contraste = random.uniform(0.7, 1.4)
        img = TF.adjust_brightness(img, factor_brillo)
        img = TF.adjust_contrast(img, factor_contraste)

    # Degradación de nitidez: simula desenfoque por movimiento o distancia
    if random.random() < self.p_nitidez:
        factor_nitidez = random.uniform(0.2, 0.9)
        img = TF.adjust_sharpness(img, factor_nitidez)

    # Posterización: simula cámaras de baja resolución de color
    if random.random() < self.p_posterizar:
        bits = random.randint(4, 7)
        img = TF.posterize(img, bits)

    return img

# Pipeline de entrenamiento con AutoAugment + TrivialAugmentWide
# RandomHorizontalFlip y RandomVerticalFlip ELIMINADOS: señales de tránsito
# no deben invertirse horizontalmente ni verticalmente (imágenes inválidas).

transform_entrenamiento = transforms.Compose([
    # Paso 1: Redimensionar a 512x512 px
    transforms.Resize((IMG_SIZE, IMG_SIZE)),

    # Paso 2: AutoAugment - política aprendida automáticamente
    autoaugment.AutoAugment(
        policy=autoaugment.AutoAugmentPolicy.IMAGENET
    ),

    # Paso 3: TrivialAugmentWide - augmentation simple y fuerte

```



```

autoaugment.TrivialAugmentWide(),

# Paso 4: Augmentaciones funcionales personalizadas (condiciones viales)
AugmentacionVial(
    p_perspectiva=0.3,
    p_exposicion=0.4,
    p_nitidez=0.3,
    p_posterizar=0.2
),

# Paso 5: Augmentaciones geométricas y de color básicas
# SIN RandomHorizontalFlip / RandomVerticalFlip (orientación inválida)
transforms.RandomRotation(degrees=20),
transforms.ColorJitter(
    brightness=0.3, contrast=0.3,
    saturation=0.2, hue=0.05),

# Paso 6: Conversión a tensor y normalización
transforms.ToTensor(),
transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])

# Pipeline de EVALUACIÓN (sin augmentation)
transform_evaluacion = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
])

print('Pipelines de transformacion creados:')
print(f' Entrenamiento: Resize({IMG_SIZE}) -> AutoAugment(ImageNet) -> TrivialAugmentWide')
print(f'                               -> AugmentacionVial (Functional) -> Rotate/ColorJitter')
print(f'                               -> ToTensor -> Normalize[-1,1]')
print(f' ELIMINADO: RandomHorizontalFlip, RandomVerticalFlip')
print(f' Evaluacion   : Resize({IMG_SIZE}) -> ToTensor -> Normalize[-1,1]')
print(f' Forma de salida: (3, {IMG_SIZE}, {IMG_SIZE})')

```

Pipelines de transformacion creados:

```

Entrenamiento: Resize(512) -> AutoAugment(ImageNet) -> TrivialAugmentWide
                -> AugmentacionVial (Functional) -> Rotate/ColorJitter
                -> ToTensor -> Normalize[-1,1]
ELIMINADO: RandomHorizontalFlip, RandomVerticalFlip
Evaluacion   : Resize(512) -> ToTensor -> Normalize[-1,1]
Forma de salida: (3, 512, 512)

```

```
[ ]: # Visualizar el efecto del data augmentation en una imagen
# Muestra la imagen original y 8 versiones aumentadas para verificar
# que las transformaciones son variadas y realistas.
# RandomHorizontalFlip y RandomVerticalFlip han sido eliminados.

import os

ruta_demo, clase_demo = datos[0]
img_original = Image.open(ruta_demo).convert('RGB')

n_versiones = 8
fig, axes = plt.subplots(2, n_versiones // 2 + 1, figsize=(18, 5))
fig.suptitle(
    f'Efecto del Data Augmentation - clase: {CLASES[clase_demo]}\n'
    f'(AutoAugment + TrivialAugmentWide + Functional Transforms | SIN Flip)',
    fontsize=12, fontweight='bold'
)

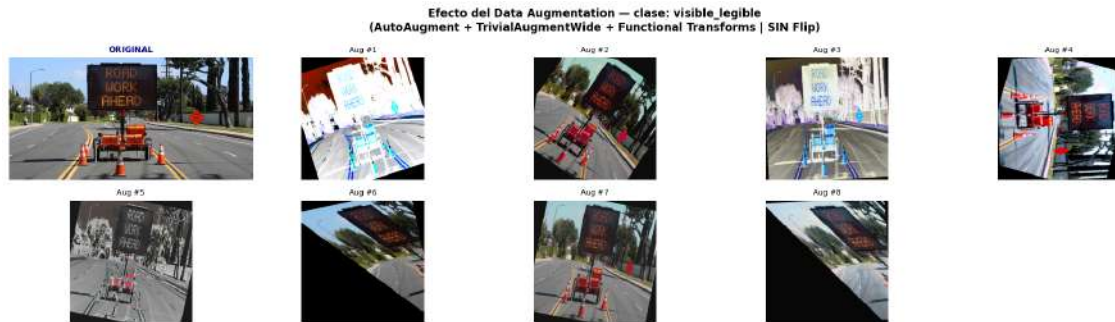
axes[0][0].imshow(img_original)
axes[0][0].set_title('ORIGINAL', fontsize=9, fontweight='bold', color='navy')
axes[0][0].axis('off')

# Pipeline de augmentation sin ToTensor/Normalize - SIN Flips
transform_visual = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    autoaugment.AutoAugment(policy=autoaugment.AutoAugmentPolicy.IMAGENET),
    autoaugment.TrivialAugmentWide(),
    AugmentacionVial(p_perspectiva=0.3, p_exposicion=0.4,
                    p_nitidez=0.3, p_posterizar=0.2),
    # RandomHorizontalFlip y RandomVerticalFlip ELIMINADOS
    transforms.RandomRotation(degrees=20),
    transforms.ColorJitter(brightness=0.3, contrast=0.3,
                          saturation=0.2, hue=0.05),
])

for i in range(n_versiones):
    fila = (i + 1) // (n_versiones // 2 + 1)
    col = (i + 1) % (n_versiones // 2 + 1)
    img_aug = transform_visual(img_original)
    axes[fila][col].imshow(img_aug)
    axes[fila][col].set_title(f'Aug #{i+1}', fontsize=8)
    axes[fila][col].axis('off')

for fila in range(2):
    for col in range(n_versiones // 2 + 1):
        if not axes[fila][col].has_data():
            axes[fila][col].set_visible(False)
```

```
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'muestra_augmentation.png'), dpi=150,
            bbox_inches='tight')
plt.show()
print('Visualizacion de augmentation guardada en Google Drive.')
```



Visualizacion de augmentation guardada en Google Drive.

1.6 4. Dataset personalizado, Cross Validation (k=5) y DataLoaders con balanceo de clases

Se utiliza **Stratified K-Fold Cross Validation** con k=5 para: - Particionar los datos de forma más robusta y reproducible. - Garantizar que cada fold conserve la proporción de clases (estratificado). - Obtener métricas promedio más confiables que una sola partición aleatoria.

El entrenamiento final usa el fold con mejor accuracy de validación, y el modelo resultante se evalúa sobre el conjunto de prueba reservado (15% del total).

```
[ ]: # Clase Dataset personalizada

from sklearn.model_selection import StratifiedKFold

class SenalesUrbanasDataset(Dataset):
    """
    Dataset para el proyecto de señalización urbana.
    Lee imágenes desde disco, las convierte a RGB y aplica transformaciones.
    """
    def __init__(self, datos, transform=None):
        self.datos = datos
        self.transform = transform

    def __len__(self):
        return len(self.datos)

    def __getitem__(self, idx):
```

```

        ruta, etiqueta = self.datos[idx]
        imagen = Image.open(ruta).convert('RGB')
        if self.transform:
            imagen = self.transform(imagen)
        return imagen, torch.tensor(etiqueta, dtype=torch.long)

# Separar conjunto de prueba (15%) antes del Cross Validation
# El conjunto de prueba queda APARTADO y no participa en ningún fold.

import numpy as np
from sklearn.model_selection import train_test_split

rutas_todas = [d[0] for d in datos]
labels_todas = [d[1] for d in datos]

# Reservar 15% para prueba final - estratificado para mantener proporción de
↳ clases
idx_trainval, idx_test = train_test_split(
    range(len(datos)),
    test_size=0.15,
    stratify=labels_todas,
    random_state=SEED
)

datos_trainval = [datos[i] for i in idx_trainval]
datos_test = [datos[i] for i in idx_test]

labels_trainval = [d[1] for d in datos_trainval]

# Dataset de prueba con transform de evaluación
test_ds = SenalesUrbanasDataset(datos_test, transform=transform_evaluacion)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False,
↳ drop_last=False)

print(f'Total de imagenes: {len(datos)}')
print(f' Train+Val : {len(datos_trainval)} imagenes (85%)')
print(f' Prueba : {len(datos_test)} imagenes (15%) - apartado hasta la
↳ evaluación final')

# Cross Validation Estratificada (k=5)
# Particiona datos_trainval en K_FOLDS folds estratificados.
# Cada fold tiene su propio train_loader y val_loader.

skf = StratifiedKFold(n_splits=K_FOLDS, shuffle=True, random_state=SEED)

```

```

folds = [] # lista de (train_loader, val_loader, fold_idx)

for fold_idx, (train_indices, val_indices) in enumerate(
    skf.split(datos_trainval, labels_trainval)):

    fold_train = [datos_trainval[i] for i in train_indices]
    fold_val    = [datos_trainval[i] for i in val_indices]

    # WeightedRandomSampler por fold para balanceo de clases
    etiquetas_fold_train = [d[1] for d in fold_train]
    conteo_fold = Counter(etiquetas_fold_train)
    pesos_fold = {cls: 1.0 / max(c, 1) for cls, c in conteo_fold.items()}
    pesos_muestras_fold = [pesos_fold[e] for e in etiquetas_fold_train]
    sampler_fold = WeightedRandomSampler(
        weights=pesos_muestras_fold,
        num_samples=len(pesos_muestras_fold),
        replacement=True
    )

    ds_train_fold = SenalesUrbanasDataset(fold_train,
    ↪transform=transform_entrenamiento)
    ds_val_fold    = SenalesUrbanasDataset(fold_val,
    ↪transform=transform_evaluacion)

    loader_train_fold = DataLoader(
        ds_train_fold, batch_size=BATCH_SIZE,
        sampler=sampler_fold, drop_last=False)
    loader_val_fold    = DataLoader(
        ds_val_fold, batch_size=BATCH_SIZE,
        shuffle=False, drop_last=False)

    folds.append((loader_train_fold, loader_val_fold, fold_idx))

    etiq_val_conteo = Counter(d[1] for d in fold_val)
    print(f' Fold {fold_idx+1}: train={len(fold_train)} val={len(fold_val)} '
          f'val_clases={dict(etiq_val_conteo)}')

print(f'\nCross Validation creada: {K_FOLDS} folds estratificados listos.')

```

Total de imagenes: 240

Train+Val : 204 imagenes (85%)

Prueba : 36 imagenes (15%) - apartado hasta la evaluación final

Fold 1: train=163 val=41 val_clases={0: 14, 2: 14, 1: 13}

Fold 2: train=163 val=41 val_clases={0: 14, 1: 14, 2: 13}

Fold 3: train=163 val=41 val_clases={2: 13, 1: 14, 0: 14}

Fold 4: train=163 val=41 val_clases={0: 13, 2: 14, 1: 14}

Fold 5: train=164 val=40 val_clases={2: 14, 0: 13, 1: 13}

Cross Validation creada: 5 folds estratificados listos.

```
[ ]: #   DataLoaders principales (fold con mejor val_acc en Cross Validation)
#   Los DataLoaders definitivos se asignan tras el bucle de CV (Sección 6).
#   Este bloque muestra la forma del batch para verificación.

# Usar el primer fold para verificación inicial de batch shape
train_loader, val_loader, _ = folds[0]

imgs_muestra, lbls_muestra = next(iter(train_loader))
print(f'Forma del batch (fold 1): {tuple(imgs_muestra.shape)}')
print(f' -> (batch={imgs_muestra.shape[0]}, canales=3, alto={IMG_SIZE},
↪ ancho={IMG_SIZE})')
print(f'Rango de valores normalizados: [{imgs_muestra.min():.2f}, {imgs_muestra.
↪ max():.2f}]')
print(f'Etiquetas del primer batch: {[CLASES[l] for l in lbls_muestra.
↪ tolist()]})')
```

Forma del batch (fold 1): (8, 3, 512, 512)

-> (batch=8, canales=3, alto=512, ancho=512)

Rango de valores normalizados: [-1.00, 1.00]

Etiquetas del primer batch: ['deteriorada', 'deteriorada', 'visible_legible',
'deteriorada', 'visible_legible', 'visible_legible', 'deteriorada',
'deteriorada']

```
[ ]: #   Visualizar imágenes YA normalizadas a 512x512x3

def desnormalizar(tensor):
    """Invierte la normalizacion [-1,1] -> [0,1] para visualizacion."""
    return torch.clamp(tensor * 0.5 + 0.5, 0, 1)

n_mostrar = min(len(imgs_muestra), 8)
fig, axes = plt.subplots(1, n_mostrar, figsize=(14, 3))
fig.suptitle(f'Imagenes normalizadas {IMG_SIZE}x{IMG_SIZE}x3 con augmentation_
↪ aplicado',
             fontsize=11, fontweight='bold')

for i, ax in enumerate(axes):
    if i >= n_mostrar:
        break
    img_vis = desnormalizar(imgs_muestra[i]).permute(1, 2, 0).numpy()
    ax.imshow(img_vis)
    ax.set_title(CLASES[lbls_muestra[i].item()], fontsize=7)
    ax.axis('off')

plt.tight_layout()
```

```
plt.show()
```



1.7 5. Arquitectura de la CNN profunda con Skip Connections (3 clases, entrada 512×512)

La arquitectura mantiene **5 bloques convolucionales** con **skip connection residual**. Con entrada 512×512, los mapas de características son más grandes, capturando más detalle.

Entrada	:	(batch, 3, 512, 512)	
Bloque 1	:	Conv(3→32, 3x3) + BN + ReLU + MaxPool	→ (batch, 32, 256, 256)
Bloque 2	:	Conv(32→64, 3x3) + BN + ReLU + MaxPool	→ (batch, 64, 128, 128)
Bloque 3	:	Conv(64→128, 3x3) + BN + ReLU	→ (batch, 128, 128, 128) ← inicio
Bloque 4	:	Conv(128→128, 3x3) + BN + ReLU + MaxPool	→ (batch, 128, 64, 64)
Bloque 5	:	Conv(128→256, 3x3) + BN + ReLU + MaxPool	→ (batch, 256, 32, 32) ← fin sk
Skip Proj.	:	Conv(128→256, 1x1) + AvgPool(4x4)	→ (batch, 256, 32, 32)
Suma residual	:	Bloque5 + Skip	→ (batch, 256, 32, 32)
AdaptiveAvgPool	:	32×32 → 8×8	→ (batch, 256, 8, 8)
Dropout	:	p=0.5	
FC1	:	256×8×8 → 512 → ReLU + Dropout(0.3)	
FC2	:	512 → 3	

- **BatchNorm**: estabiliza y acelera el entrenamiento
- **Skip Connection**: evita que gradientes desaparezcan en capas profundas
- **AdaptiveAvgPool**: reduce espacialmente de forma adaptativa sin fijar tamaño
- **Doble Dropout**: regularización fuerte para datasets de tamaño moderado
- **CrossEntropyLoss con pesos**: penaliza más errores en clases con menos ejemplos
- **3 clases**: visible_legible, deteriorada, poco_visible

```
[ ]: # CNN profunda con skip connection residual para 512×512×3

class CNN_SenalesUrbanas(nn.Module):
    """
    CNN de 5 bloques convolucionales con skip connection para clasificar
    3 clases de señalización urbana con entrada de 512×512×3.

    Mejoras v5:
    - Entrada 512×512 para mayor detalle visual
    - AdaptiveAvgPool para manejar cualquier tamaño de entrada
    - 3 clases de salida
```

```

- Sin RandomHorizontalFlip / RandomVerticalFlip
"""
def __init__(self, num_clases=3):
    super().__init__()

    # Bloques convolucionales
    self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
    self.bn1 = nn.BatchNorm2d(32)

    self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
    self.bn2 = nn.BatchNorm2d(64)

    self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
    self.bn3 = nn.BatchNorm2d(128)

    self.conv4 = nn.Conv2d(128, 128, 3, padding=1)
    self.bn4 = nn.BatchNorm2d(128)

    self.conv5 = nn.Conv2d(128, 256, 3, padding=1)
    self.bn5 = nn.BatchNorm2d(256)

    # Skip connection: proyección bloque 3 -> dimensión bloque 5
    self.skip_proj = nn.Sequential(
        nn.Conv2d(128, 256, kernel_size=1, bias=False),
        nn.AvgPool2d(kernel_size=4, stride=4)
    )

    self.pool = nn.MaxPool2d(2, 2)

    # AdaptiveAvgPool: reduce a 8x8 independientemente del tamaño de entrada
    self.adaptive_pool = nn.AdaptiveAvgPool2d((8, 8))

    self.dropout_conv = nn.Dropout(p=0.5)
    self.dropout_fc = nn.Dropout(p=0.3)

    # Capas fully connected
    self.fc1 = nn.Linear(256 * 8 * 8, 512) # 16384 -> 512
    self.fc2 = nn.Linear(512, num_clases) # 512 -> 3

def forward(self, x):
    # Bloques 1-2
    x = self.pool(F.relu(self.bn1(self.conv1(x))))
    x = self.pool(F.relu(self.bn2(self.conv2(x))))

    # Bloque 3: inicio skip connection
    x = F.relu(self.bn3(self.conv3(x)))
    skip = x

```



```

    # Bloques 4-5
    x = self.pool(F.relu(self.bn4(self.conv4(x))))
    x = self.pool(F.relu(self.bn5(self.conv5(x))))

    # Suma residual
    skip_proyectado = self.skip_proj(skip)
    x = x + skip_proyectado
    x = F.relu(x)

    # Reducción adaptativa a 8x8
    x = self.adaptive_pool(x)

    # Clasificación
    x = self.dropout_conv(x)
    x = x.view(x.size(0), -1)          # -> (B, 16384)
    x = self.dropout_fc(F.relu(self.fc1(x))) # -> (B, 512)
    x = self.fc2(x)                    # -> (B, 3)
    return x

# Verificar dimensiones con una pasada de prueba
model = CNN_SenalesUrbanas(num_clases=NUM_CLASES).to(device)

with torch.no_grad():
    x_test = torch.randn(1, 3, IMG_SIZE, IMG_SIZE).to(device)
    out = model(x_test)
    print(f'Entrada: {tuple(x_test.shape)} -> Salida: {tuple(out.shape)}')

print('\nArquitectura CNN profunda con skip connection (3 clases, 512x512):')
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Parametros entrenables: {total_params:,}')

```

Entrada: (1, 3, 512, 512) -> Salida: (1, 3)

Arquitectura CNN profunda con skip connection (3 clases, 512x512):
 Parametros entrenables: 8,960,643

1.8 6. Función de pérdida, optimizador y entrenamiento

```

[ ]: import torch
    from collections import Counter
    import torch.nn as nn
    import torch.optim as optim

    # CrossEntropyLoss con pesos por clase
    # Con 23 imgs en visible_legible vs 12 en deteriorada, el modelo sin pesos

```

```

# tendería a ignorar la clase minoritaria.
# Peso = total_imgs / (num_clases * conteo_clase) → clases pequeñas = mayor peso
# peso

conteo_clases = Counter(lbl for _, lbl in datos)
total_imgs = len(datos)
pesos_clase = torch.tensor(
    [total_imgs / (NUM_CLASES * max(conteo_clases[i], 1)) for i in
    range(NUM_CLASES)],
    dtype=torch.float
).to(device)

criterion = nn.CrossEntropyLoss(weight=pesos_clase)

# Adam con weight decay para regularización
optimizer = optim.Adam(model.parameters(), lr=5e-4, weight_decay=2e-4)

# Scheduler: reduce lr cuando la pérdida de validación se estanca
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=10
)

# Gradient Clipping: evita gradientes explosivos
# Con redes más profundas (5 bloques), los gradientes pueden crecer sin control.
# max_norm=1.0 recorta el gradiente si su norma supera 1.0.
MAX_GRAD_NORM = 1.0

print('Configuracion del entrenamiento:')
print(f' Funcion de perdida : CrossEntropyLoss con pesos de clase')
print(f' Pesos por clase : {[f"{p:.2f}" for p in pesos_clase.cpu().
    tolist()]}')
print(f' Optimizador : Adam lr=5e-4 weight_decay=2e-4')
print(f' Scheduler : ReduceLROnPlateau (factor=0.5, patience=10)')
print(f' Gradient Clipping : max_norm={MAX_GRAD_NORM}')
print(f' Epocas : {NUM_EPOCHS}')
print(f' Batch size : {BATCH_SIZE}')

```

Configuracion del entrenamiento:

```

Funcion de perdida : CrossEntropyLoss con pesos de clase
Pesos por clase : ['1.00', '1.00', '1.00']
Optimizador : Adam lr=5e-4 weight_decay=2e-4
Scheduler : ReduceLROnPlateau (factor=0.5, patience=10)
Gradient Clipping : max_norm=1.0
Epocas : 120
Batch size : 8

```

```
[ ]: # Bucle de entrenamiento con Cross Validation (k=5)
# Para cada fold: entrena 120 épocas, registra métricas.
# Al final se selecciona el fold con mejor val_acc y se guarda ese modelo.

import copy

resultados_cv = [] # métricas de cada fold
mejor_val_acc = -1
mejor_fold_idx = -1
mejor_estado = None

for loader_train_fold, loader_val_fold, fold_idx in folds:

    print(f'\n{"="*85}')
    print(f'FOLD {fold_idx+1}/{K_FOLDS} - {NUM_EPOCHS} épocas')
    print(f'{"="*85}')

    # Reinstanciar modelo y optimizador para cada fold
    model_fold = CNN_SenalesUrbanas(num_clases=NUM_CLASES).to(device)

    # CrossEntropyLoss con pesos por clase
    conteo_clases_fold = Counter(lbl for _, lbl in datos)
    total_fold = len(datos)
    pesos_fold_tensor = torch.tensor(
        [total_fold / (NUM_CLASES * max(conteo_clases_fold[i], 1)) for i in
↪range(NUM_CLASES)],
        dtype=torch.float
    ).to(device)
    criterion_fold = nn.CrossEntropyLoss(weight=pesos_fold_tensor)

    optimizer_fold = optim.Adam(model_fold.parameters(), lr=5e-4,
↪weight_decay=2e-4)
    scheduler_fold = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer_fold, mode='min', factor=0.5, patience=10)

    historial_fold = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc':
↪[]}
    mejor_val_loss_fold = float('inf')

    for epoch in range(NUM_EPOCHS):
        t0 = time.time()

        # ENTRENAMIENTO
        model_fold.train()
        running_loss, correct, total = 0.0, 0, 0
        for imgs, labels in loader_train_fold:
            imgs, labels = imgs.to(device), labels.to(device)
```

```

optimizer_fold.zero_grad()
outputs = model_fold(imgs)
loss = criterion_fold(outputs, labels)
loss.backward()
torch.nn.utils.clip_grad_norm_(model_fold.parameters(),
↪MAX_GRAD_NORM)
optimizer_fold.step()
running_loss += loss.item()
_, preds = torch.max(outputs, 1)
correct += (preds == labels).sum().item()
total += labels.size(0)

train_loss = running_loss / len(loader_train_fold)
train_acc = 100.0 * correct / total

# VALIDACIÓN
model_fold.eval()
v_loss, v_correct, v_total = 0.0, 0, 0
with torch.no_grad():
    for imgs, labels in loader_val_fold:
        imgs, labels = imgs.to(device), labels.to(device)
        outputs = model_fold(imgs)
        loss = criterion_fold(outputs, labels)
        v_loss += loss.item()
        _, preds = torch.max(outputs, 1)
        v_correct += (preds == labels).sum().item()
        v_total += labels.size(0)

val_loss = v_loss / max(len(loader_val_fold), 1)
val_acc = 100.0 * v_correct / max(v_total, 1)
scheduler_fold.step(val_loss)

if val_loss < mejor_val_loss_fold:
    mejor_val_loss_fold = val_loss

historial_fold['train_loss'].append(train_loss)
historial_fold['val_loss'].append(val_loss)
historial_fold['train_acc'].append(train_acc)
historial_fold['val_acc'].append(val_acc)

if (epoch + 1) % 10 == 0 or epoch == 0:
    dur = time.time() - t0
    lr_actual = optimizer_fold.param_groups[0]['lr']
    print(f' Ep {epoch+1:3d}/{NUM_EPOCHS} | '
          f'Train Loss:{train_loss:.4f} Acc:{train_acc:.1f}% | '
          f'Val Loss:{val_loss:.4f} Acc:{val_acc:.1f}% | '
          f'lr:{lr_actual:.5f} | {dur:.1f}s')

```

```

# Mejor val_acc de este fold (último epoch)
fold_val_acc_max = max(historial_fold['val_acc'])
resultados_cv.append({
    'fold': fold_idx + 1,
    'val_acc_max': fold_val_acc_max,
    'historial': historial_fold
})
print(f' Fold {fold_idx+1} terminado - mejor val_acc: {fold_val_acc_max:.
↪2f}%')

# Guardar estado del mejor fold
if fold_val_acc_max > mejor_val_acc:
    mejor_val_acc = fold_val_acc_max
    mejor_fold_idx = fold_idx + 1
    mejor_estado = copy.deepcopy(model_fold.state_dict())
    historial      = historial_fold # historial del mejor fold para ↪
↪graficas
    model          = model_fold    # referencia al mejor modelo

# Guardar el mejor modelo
torch.save(mejor_estado, RUTA_MODELO)
model.load_state_dict(mejor_estado)

print(f'\n{"="*85}')
print(f'Cross Validation finalizado.')
print(f' Resultados por fold:')
for r in resultados_cv:
    print(f' Fold {r["fold"]}: mejor val_acc = {r["val_acc_max"]:.2f}%')
val_accs = [r['val_acc_max'] for r in resultados_cv]
print(f' Promedio val_acc: {np.mean(val_accs):.2f}% ± {np.std(val_accs):.2f}%')
print(f' Mejor fold: {mejor_fold_idx} val_acc={mejor_val_acc:.2f}%')
print(f' Modelo guardado en: {RUTA_MODELO}')

```

```

=====
=====
FOLD 1/5 - 120 épocas
=====
=====
Ep   1/120 | Train Loss:2.2456 Acc:38.7% | Val Loss:1.2121 Acc:31.7% |
lr:0.00050 | 72.3s
Ep  10/120 | Train Loss:1.0052 Acc:50.3% | Val Loss:1.1796 Acc:48.8% |
lr:0.00050 | 15.5s
Ep  20/120 | Train Loss:0.9197 Acc:55.2% | Val Loss:1.2600 Acc:51.2% |
lr:0.00025 | 16.8s
Ep  30/120 | Train Loss:0.9540 Acc:55.2% | Val Loss:1.1470 Acc:63.4% |

```

```

lr:0.00025 | 15.4s
  Ep  40/120 | Train Loss:0.8107 Acc:62.6% | Val Loss:0.9730 Acc:56.1% |
lr:0.00013 | 15.6s
  Ep  50/120 | Train Loss:0.7776 Acc:66.9% | Val Loss:0.9786 Acc:61.0% |
lr:0.00006 | 15.5s
  Ep  60/120 | Train Loss:0.7310 Acc:65.0% | Val Loss:0.8658 Acc:56.1% |
lr:0.00003 | 15.2s
  Ep  70/120 | Train Loss:0.6926 Acc:71.8% | Val Loss:1.0524 Acc:58.5% |
lr:0.00002 | 14.9s
  Ep  80/120 | Train Loss:0.7455 Acc:67.5% | Val Loss:0.9789 Acc:56.1% |
lr:0.00002 | 15.1s
  Ep  90/120 | Train Loss:0.7420 Acc:66.9% | Val Loss:0.9850 Acc:56.1% |
lr:0.00001 | 16.3s
  Ep 100/120 | Train Loss:0.6227 Acc:75.5% | Val Loss:0.9783 Acc:58.5% |
lr:0.00000 | 15.4s
  Ep 110/120 | Train Loss:0.6789 Acc:71.2% | Val Loss:0.9771 Acc:56.1% |
lr:0.00000 | 16.0s
  Ep 120/120 | Train Loss:0.6721 Acc:67.5% | Val Loss:0.9903 Acc:58.5% |
lr:0.00000 | 15.1s
  Fold 1 terminado - mejor val_acc: 70.73%

```

```

=====
=====
FOLD 2/5 - 120 épocas
=====
=====

```

```

  Ep   1/120 | Train Loss:2.3183 Acc:38.0% | Val Loss:1.4130 Acc:41.5% |
lr:0.00050 | 15.6s
  Ep  10/120 | Train Loss:1.0076 Acc:49.7% | Val Loss:0.8781 Acc:43.9% |
lr:0.00050 | 15.2s
  Ep  20/120 | Train Loss:1.0462 Acc:53.4% | Val Loss:0.9701 Acc:56.1% |
lr:0.00050 | 15.3s
  Ep  30/120 | Train Loss:0.8675 Acc:62.0% | Val Loss:0.8583 Acc:51.2% |
lr:0.00025 | 15.6s
  Ep  40/120 | Train Loss:0.7743 Acc:65.6% | Val Loss:1.1121 Acc:53.7% |
lr:0.00025 | 15.2s
  Ep  50/120 | Train Loss:0.8539 Acc:63.8% | Val Loss:1.1715 Acc:48.8% |
lr:0.00013 | 15.9s
  Ep  60/120 | Train Loss:0.7970 Acc:71.8% | Val Loss:1.0198 Acc:58.5% |
lr:0.00006 | 15.2s
  Ep  70/120 | Train Loss:0.7623 Acc:66.3% | Val Loss:0.9855 Acc:53.7% |
lr:0.00003 | 15.6s
  Ep  80/120 | Train Loss:0.6864 Acc:73.6% | Val Loss:1.0918 Acc:53.7% |
lr:0.00002 | 16.1s
  Ep  90/120 | Train Loss:0.6855 Acc:66.3% | Val Loss:1.0326 Acc:53.7% |
lr:0.00001 | 16.0s
  Ep 100/120 | Train Loss:0.7928 Acc:65.0% | Val Loss:1.0360 Acc:56.1% |
lr:0.00000 | 15.4s

```

```

    Ep 110/120 | Train Loss:0.6658 Acc:69.9% | Val Loss:1.0861 Acc:53.7% |
lr:0.00000 | 15.4s
    Ep 120/120 | Train Loss:0.6292 Acc:72.4% | Val Loss:1.0906 Acc:53.7% |
lr:0.00000 | 15.6s
    Fold 2 terminado - mejor val_acc: 60.98%

```

```

=====
=====
FOLD 3/5 - 120 épocas
=====
=====

```

```

    Ep  1/120 | Train Loss:2.7426 Acc:35.0% | Val Loss:1.6669 Acc:34.1% |
lr:0.00050 | 16.7s
    Ep 10/120 | Train Loss:1.0278 Acc:47.2% | Val Loss:1.1993 Acc:43.9% |
lr:0.00050 | 15.6s
    Ep 20/120 | Train Loss:0.9399 Acc:58.9% | Val Loss:1.9930 Acc:53.7% |
lr:0.00050 | 15.3s
    Ep 30/120 | Train Loss:0.9665 Acc:54.0% | Val Loss:0.8610 Acc:65.9% |
lr:0.00025 | 15.4s
    Ep 40/120 | Train Loss:0.8609 Acc:63.2% | Val Loss:1.1383 Acc:58.5% |
lr:0.00025 | 14.8s
    Ep 50/120 | Train Loss:0.9165 Acc:58.3% | Val Loss:0.9146 Acc:51.2% |
lr:0.00025 | 15.1s
    Ep 60/120 | Train Loss:0.8146 Acc:63.2% | Val Loss:0.8254 Acc:56.1% |
lr:0.00013 | 15.6s
    Ep 70/120 | Train Loss:0.5815 Acc:73.6% | Val Loss:0.8544 Acc:53.7% |
lr:0.00013 | 16.0s
    Ep 80/120 | Train Loss:0.6966 Acc:73.0% | Val Loss:0.8458 Acc:58.5% |
lr:0.00006 | 15.5s
    Ep 90/120 | Train Loss:0.6224 Acc:74.8% | Val Loss:0.8455 Acc:61.0% |
lr:0.00003 | 15.8s
    Ep 100/120 | Train Loss:0.7329 Acc:71.2% | Val Loss:0.9220 Acc:61.0% |
lr:0.00002 | 14.7s
    Ep 110/120 | Train Loss:0.6692 Acc:71.8% | Val Loss:0.8610 Acc:58.5% |
lr:0.00001 | 15.2s
    Ep 120/120 | Train Loss:0.6370 Acc:69.9% | Val Loss:0.8094 Acc:56.1% |
lr:0.00000 | 17.5s
    Fold 3 terminado - mejor val_acc: 68.29%

```

```

=====
=====
FOLD 4/5 - 120 épocas
=====
=====

```

```

    Ep  1/120 | Train Loss:2.8294 Acc:32.5% | Val Loss:1.2279 Acc:31.7% |
lr:0.00050 | 15.2s
    Ep 10/120 | Train Loss:1.0825 Acc:42.3% | Val Loss:1.0844 Acc:53.7% |
lr:0.00050 | 15.4s

```

```

    Ep 20/120 | Train Loss:0.9774 Acc:55.8% | Val Loss:0.9806 Acc:61.0% |
lr:0.00050 | 15.6s
    Ep 30/120 | Train Loss:0.8766 Acc:60.1% | Val Loss:1.0649 Acc:58.5% |
lr:0.00050 | 15.0s
    Ep 40/120 | Train Loss:0.7959 Acc:62.6% | Val Loss:1.0848 Acc:61.0% |
lr:0.00025 | 16.6s
    Ep 50/120 | Train Loss:0.8397 Acc:65.6% | Val Loss:1.1275 Acc:61.0% |
lr:0.00013 | 17.1s
    Ep 60/120 | Train Loss:0.7305 Acc:70.6% | Val Loss:1.4046 Acc:63.4% |
lr:0.00006 | 15.4s
    Ep 70/120 | Train Loss:0.7470 Acc:67.5% | Val Loss:1.3742 Acc:61.0% |
lr:0.00003 | 14.9s
    Ep 80/120 | Train Loss:0.7351 Acc:66.3% | Val Loss:1.4538 Acc:61.0% |
lr:0.00002 | 15.8s
    Ep 90/120 | Train Loss:0.6767 Acc:68.7% | Val Loss:1.4822 Acc:61.0% |
lr:0.00002 | 14.9s
    Ep 100/120 | Train Loss:0.6438 Acc:71.2% | Val Loss:1.4111 Acc:61.0% |
lr:0.00001 | 16.9s
    Ep 110/120 | Train Loss:0.6961 Acc:68.1% | Val Loss:1.5362 Acc:61.0% |
lr:0.00000 | 14.6s
    Ep 120/120 | Train Loss:0.6883 Acc:69.3% | Val Loss:1.4025 Acc:61.0% |
lr:0.00000 | 16.3s
    Fold 4 terminado - mejor val_acc: 70.73%

```

```

=====
=====
FOLD 5/5 - 120 épocas
=====
=====
    Ep 1/120 | Train Loss:2.4880 Acc:34.8% | Val Loss:1.0952 Acc:40.0% |
lr:0.00050 | 16.0s
    Ep 10/120 | Train Loss:1.0457 Acc:45.7% | Val Loss:1.2467 Acc:40.0% |
lr:0.00050 | 15.1s
    Ep 20/120 | Train Loss:1.0201 Acc:51.2% | Val Loss:1.0034 Acc:42.5% |
lr:0.00050 | 15.1s
    Ep 30/120 | Train Loss:0.9356 Acc:58.5% | Val Loss:1.0207 Acc:55.0% |
lr:0.00050 | 15.9s
    Ep 40/120 | Train Loss:0.9510 Acc:59.8% | Val Loss:0.9319 Acc:55.0% |
lr:0.00025 | 15.7s
    Ep 50/120 | Train Loss:0.7143 Acc:65.2% | Val Loss:0.9426 Acc:55.0% |
lr:0.00025 | 15.8s
    Ep 60/120 | Train Loss:0.7950 Acc:65.2% | Val Loss:1.0011 Acc:52.5% |
lr:0.00013 | 16.7s
    Ep 70/120 | Train Loss:0.7896 Acc:61.0% | Val Loss:0.8782 Acc:52.5% |
lr:0.00006 | 15.1s
    Ep 80/120 | Train Loss:0.6150 Acc:76.2% | Val Loss:0.8894 Acc:55.0% |
lr:0.00006 | 15.6s
    Ep 90/120 | Train Loss:0.6092 Acc:70.7% | Val Loss:0.9831 Acc:55.0% |

```



```

lr:0.00003 | 16.2s
  Ep 100/120 | Train Loss:0.6774 Acc:72.6% | Val Loss:0.9546 Acc:55.0% |
lr:0.00002 | 14.7s
  Ep 110/120 | Train Loss:0.6154 Acc:75.0% | Val Loss:1.0054 Acc:52.5% |
lr:0.00001 | 15.0s
  Ep 120/120 | Train Loss:0.5757 Acc:76.2% | Val Loss:0.9308 Acc:55.0% |
lr:0.00000 | 14.6s
  Fold 5 terminado - mejor val_acc: 60.00%

```

```

=====
=====

```

Cross Validation finalizado.

Resultados por fold:

Fold 1: mejor val_acc = 70.73%

Fold 2: mejor val_acc = 60.98%

Fold 3: mejor val_acc = 68.29%

Fold 4: mejor val_acc = 70.73%

Fold 5: mejor val_acc = 60.00%

Promedio val_acc: 66.15% ± 4.72%

Mejor fold: 1 val_acc=70.73%

Modelo guardado en:

/content/drive/MyDrive/SenalesTransito/modelo_3clases_v5.pth

```

[ ]: # Gráficas de evolución del entrenamiento
# Permite detectar sobreajuste (overfitting): si train_acc sube pero val_acc
↳ baja

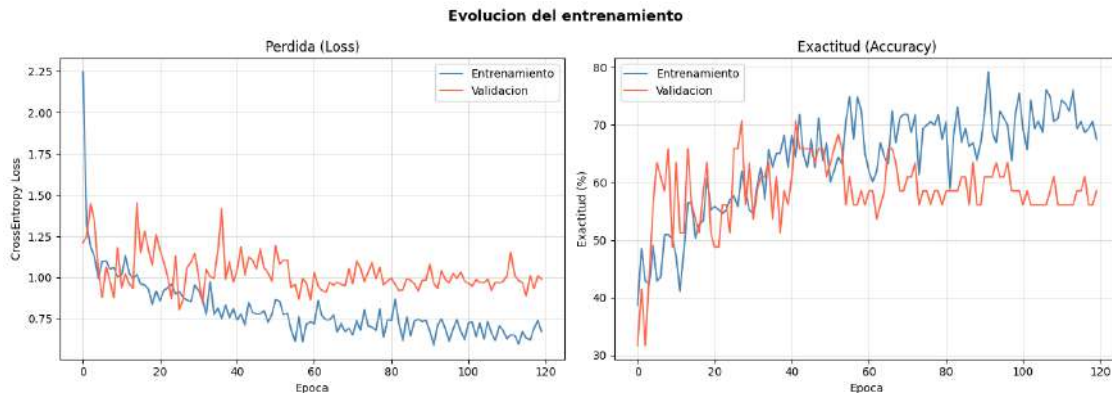
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Evolucion del entrenamiento', fontsize=13, fontweight='bold')

# Gráfica de pérdida
axes[0].plot(historial['train_loss'], label='Entrenamiento', color='steelblue')
axes[0].plot(historial['val_loss'], label='Validacion', color='tomato')
axes[0].set_title('Pérdida (Loss)')
axes[0].set_xlabel('Epoca')
axes[0].set_ylabel('CrossEntropy Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.4)

# Gráfica de exactitud
axes[1].plot(historial['train_acc'], label='Entrenamiento', color='steelblue')
axes[1].plot(historial['val_acc'], label='Validacion', color='tomato')
axes[1].set_title('Exactitud (Accuracy)')
axes[1].set_xlabel('Epoca')
axes[1].set_ylabel('Exactitud (%)')
axes[1].legend()
axes[1].grid(True, alpha=0.4)

```

```
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'metricas_entrenamiento.png'), dpi=150,
            bbox_inches='tight')
plt.show()
print('Grafica guardada en Google Drive.')
```



Grafica guardada en Google Drive.

1.9 7. Evaluación unificada — Matrices de confusión (Train y Test)

Se carga el mejor modelo del fold con mayor val_acc y se evalúa sobre: - **Conjunto de entrenamiento**: para verificar que el modelo aprende correctamente las clases. - **Conjunto de prueba**: para medir generalización real del modelo.

La **matriz de confusión** muestra para cada clase real cuántas imágenes fueron clasificadas correctamente (diagonal) y cuántas fueron confundidas con otras clases. Se espera alta concentración en la diagonal gracias al balanceo de clases y la eliminación de los Flips que introducían ambigüedad.

```
[ ]: # Cargar el mejor modelo y evaluar en Test y Train

model.load_state_dict(torch.load(RUTA_MODELO, map_location=device))
model.eval()
print(f'Mejor modelo (fold {mejor_fold_idx}) cargado desde: {RUTA_MODELO}')
```

Evaluación en PRUEBA (Test)

```
all_preds_test, all_labels_test = [], []
with torch.no_grad():
    for imgs, labels in test_loader:
        imgs = imgs.to(device)
        outputs = model(imgs)
        _, preds = torch.max(outputs, 1)
        all_preds_test.extend(preds.cpu().numpy())
        all_labels_test.extend(labels.numpy())
```

```

acc_test = 100.0 * sum(p == 1 for p, l in zip(all_preds_test, all_labels_test)) / len(all_labels_test)
print(f'Exactitud global en PRUEBA: {acc_test:.1f}%')

# Evaluación en ENTRENAMIENTO (Train)
# Usar el DataLoader del mejor fold sin augmentation para evaluación pura
ds_train_eval = SenalesUrbanasDataset(
    [datos_trainval[i] for i in range(len(datos_trainval))],
    transform=transform_evaluacion
)
train_eval_loader = DataLoader(
    ds_train_eval, batch_size=BATCH_SIZE, shuffle=False, drop_last=False)

all_preds_train, all_labels_train = [], []
with torch.no_grad():
    for imgs, labels in train_eval_loader:
        imgs = imgs.to(device)
        outputs = model(imgs)
        _, preds = torch.max(outputs, 1)
        all_preds_train.extend(preds.cpu().numpy())
        all_labels_train.extend(labels.numpy())

acc_train = 100.0 * sum(p == 1 for p, l in zip(all_preds_train, all_labels_train)) / len(all_labels_train)
print(f'Exactitud global en ENTRENAMIENTO: {acc_train:.1f}%')

```

Mejor modelo (fold 1) cargado desde:
 /content/drive/MyDrive/SenalesTransito/modelo_3clases_v5.pth
 Exactitud global en PRUEBA: 44.4%
 Exactitud global en ENTRENAMIENTO: 81.9%

```

[ ]: # Matrices de confusión: Test y Train

etiquetas_cortas = [
    'Visible\nLegible',
    'Deteriorada',
    'Poco\nVisible'
]

def plot_confusion_matrix(cm, titulo, acc, ax):
    """Dibuja una matriz de confusión con diagonal resaltada."""
    sns.heatmap(
        cm,
        annot=True, fmt='d', cmap='Blues',
        xticklabels=etiquetas_cortas,
        yticklabels=etiquetas_cortas,
    )

```

```

        linewidths=1.5, linecolor='white',
        annot_kws={'size': 14, 'weight': 'bold'},
        ax=ax
    )
    for i in range(len(cm)):
        ax.add_patch(plt.Rectangle(
            (i, i), 1, 1,
            fill=False, edgecolor='black', linewidth=3, clip_on=False
        ))
    ax.set_xlabel('Clase Predicha', fontsize=11, labelpad=8)
    ax.set_ylabel('Clase Real',      fontsize=11, labelpad=8)
    ax.set_title(f'{titulo}\nExactitud: {acc:.1f}%', fontsize=12,
        ↪fontweight='bold')

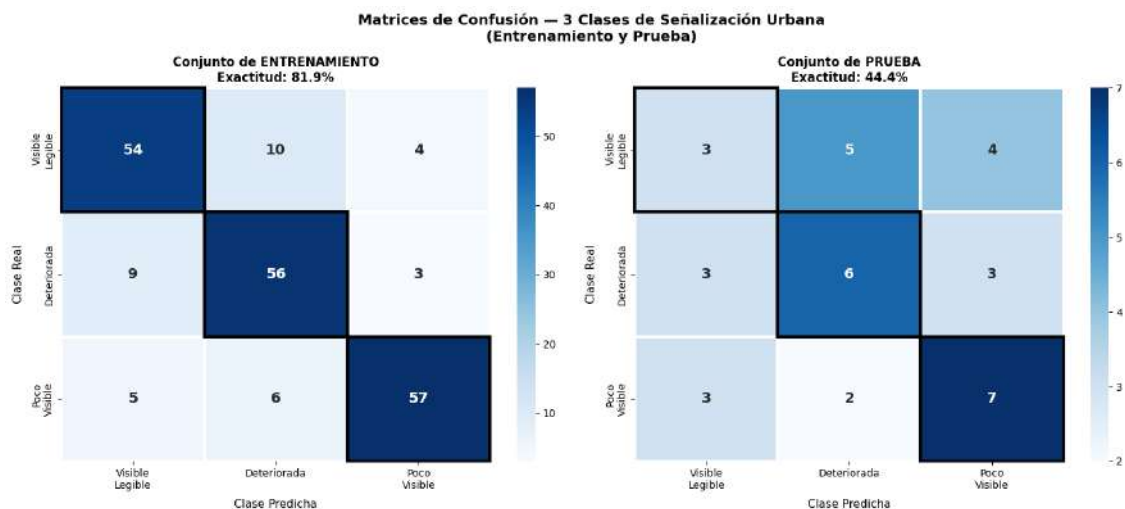
cm_test  = confusion_matrix(all_labels_test, all_preds_test)
cm_train = confusion_matrix(all_labels_train, all_preds_train)

fig, axes = plt.subplots(1, 2, figsize=(16, 7))
fig.suptitle(
    'Matrices de Confusión - 3 Clases de Señalización Urbana\n'
    '(Entrenamiento y Prueba)',
    fontsize=14, fontweight='bold'
)

plot_confusion_matrix(cm_train, 'Conjunto de ENTRENAMIENTO', acc_train, axes[0])
plot_confusion_matrix(cm_test,  'Conjunto de PRUEBA',      acc_test,  axes[1])

plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'matrices_confusion_train_test.png'),
    ↪dpi=150, bbox_inches='tight')
plt.show()
print('Matrices de confusion (Train + Test) guardadas en Google Drive.')

```



Matrices de confusion (Train + Test) guardadas en Google Drive.

```
[ ]: # Reporte de clasificación por clase (Prueba)

print('Reporte de Clasificación por Clase - Conjunto de PRUEBA:')
print('=' * 70)
print(classification_report(
    all_labels_test, all_preds_test,
    target_names=CLASES,
    labels=range(len(CLASES)),
    zero_division=0
))

print('Reporte de Clasificación por Clase - Conjunto de ENTRENAMIENTO:')
print('=' * 70)
print(classification_report(
    all_labels_train, all_preds_train,
    target_names=CLASES,
    labels=range(len(CLASES)),
    zero_division=0
))
```

Reporte de Clasificación por Clase - Conjunto de PRUEBA:

```
=====
              precision    recall  f1-score   support

visible_legible      0.33      0.25      0.29         12
deteriorada          0.46      0.50      0.48         12
poco_visible          0.50      0.58      0.54         12
```

accuracy			0.44	36
macro avg	0.43	0.44	0.43	36
weighted avg	0.43	0.44	0.43	36

Reporte de Clasificación por Clase - Conjunto de ENTRENAMIENTO:

	precision	recall	f1-score	support
visible_legible	0.79	0.79	0.79	68
deteriorada	0.78	0.82	0.80	68
poco_visible	0.89	0.84	0.86	68
accuracy			0.82	204
macro avg	0.82	0.82	0.82	204
weighted avg	0.82	0.82	0.82	204

1.10 8. Análisis individual por clase

Esta sección muestra, para cada una de las 3 clases, qué imágenes del conjunto de prueba fueron **correctamente clasificadas** y cuáles fueron **confundidas**, permitiendo entender las fortalezas y debilidades del modelo por categoría.

Gracias a la eliminación de Flips y al Cross Validation con k=5, se espera mayor concentración en la diagonal y menor dispersión entre clases.

```
[ ]: # Visualización de aciertos y errores por clase (Prueba)

rutas_test = [datos[i][0] for i in idx_test]
etiq_reales = all_labels_test
etiq_preds = all_preds_test

MAX_MUESTRA = 4

for clase_idx, clase_nombre in enumerate(CLASES):
    aciertos = [(r, p) for r, real, p in zip(rutas_test, etiq_reales,
↪ etiq_preds)
                if real == clase_idx and p == clase_idx]
    errores = [(r, p) for r, real, p in zip(rutas_test, etiq_reales,
↪ etiq_preds)
              if real == clase_idx and p != clase_idx]

    n_a = min(len(aciertos), MAX_MUESTRA)
    n_e = min(len(errores), MAX_MUESTRA)
    total_cols = n_a + n_e
    if total_cols == 0:
        continue
```

```

fig, axes = plt.subplots(1, total_cols + 1, figsize=(2.5 * (total_cols + 1), 3))
fig.suptitle(f'Clase: {clase_nombre} | Aciertos: {len(aciertos)} Errores: {len(errores)}',
             fontsize=10, fontweight='bold')

axes[0].text(0.5, 0.5, clase_nombre.replace('_', '\n'),
            ha='center', va='center', fontsize=9, fontweight='bold',
            color='navy', transform=axes[0].transAxes)
axes[0].axis('off')

col = 1
for ruta, pred in aciertos[:n_a]:
    img = Image.open(ruta).convert('RGB')
    axes[col].imshow(img)
    axes[col].set_title('OK', color='green', fontsize=8)
    axes[col].axis('off')
    col += 1

for ruta, pred in errores[:n_e]:
    img = Image.open(ruta).convert('RGB')
    axes[col].imshow(img)
    axes[col].set_title(f'X -> {CLASES[pred]}', color='red', fontsize=7)
    axes[col].axis('off')
    col += 1

for j in range(col, len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.show()

```





1.11 9. Poner el modelo a prueba — Clasificar una nueva imagen

Sube cualquier imagen de señalización urbana. El modelo indicará: - La **clase predicha** de las 4 posibles - El **porcentaje de confianza** de cada clase - Las **métricas de calidad** de la imagen (brillo, contraste, nitidez) - Un **diagnóstico detallado** de los defectos presentes

Se ofrecen **dos opciones**: subir desde tu PC o usar una imagen de Google Drive.

```
[ ]: # Función principal de análisis y diagnóstico

def analizar_imagen(ruta_imagen):
    """
    Clasifica una imagen de señalización urbana con el modelo CNN entrenado
    y genera un diagnóstico detallado basado en métricas de calidad.

    Args:
        ruta_imagen (str): ruta local de la imagen a analizar

    Returns:
        dict con clase, confianza, probabilidades, métricas y defectos
    """
    # 1. Métricas de calidad visual (OpenCV)
    img_cv = cv2.imread(ruta_imagen)
    img_cv = cv2.cvtColor(img_cv, cv2.COLOR_BGR2RGB)
    gris = cv2.cvtColor(img_cv, cv2.COLOR_RGB2GRAY)

    brillo = float(np.mean(gris)) # brillo
    ↪promedio [0,255]
    contraste = float(np.std(gris)) # desviación
    ↪estándar de intensidad
    nitidez = float(cv2.Laplacian(gris, cv2.CV_64F).var()) # varianza del
    ↪Laplaciano

    # 2. Predicción con la CNN
    img_pil = Image.open(ruta_imagen).convert('RGB')
    tensor = transform_evaluacion(img_pil).unsqueeze(0).to(device) #
    ↪(1,3,64,64)

    model.eval()
    with torch.no_grad():
```



```

logits = model(tensor)
probs = F.softmax(logits, dim=1)[0] # probabilidades por clase
pred = torch.argmax(probs).item()

clase_pred = CLASES[pred]
confianza = float(probs[pred]) * 100

# 3. Diagnóstico de defectos según clase y métricas
defectos = []

# Defectos relacionados con iluminación
if brillo < 70:
    defectos.append(f'Illuminacion muy baja (brillo={brillo:.1f}/255):  

    ↪ posible mala iluminacion nocturna o sombras')
elif brillo > 220:
    defectos.append(f'Sobreexposicion (brillo={brillo:.1f}/255):  

    ↪ deslumbramiento o reflejo solar directo')

# Defectos relacionados con contraste
if contraste < 25:
    defectos.append(f'Contraste bajo (sigma={contraste:.1f}): colores de la  

    ↪ señal no se diferencian bien')

# Defectos relacionados con nitidez
if nitidez < 40:
    defectos.append(f'Imagen muy borrosa (nitidez={nitidez:.1f}): angulo  

    ↪ inadecuado, distancia excesiva o movimiento')
elif nitidez < 120:
    defectos.append(f'Nitidez moderada (nitidez={nitidez:.1f}): posible  

    ↪ lejanía o leve desenfoque')

# Defectos según la clase predicha
diagnosticos_clase = {
    'deteriorada': 'La señal presenta daños físicos: pintura desgastada,  

    ↪ oxido, deformación o grafiti',
    'obstruida': 'La señal está tapada parcial o totalmente por árboles,  

    ↪ cables, postes u objetos',
    'poco_visible': 'La señal no es legible por mala iluminación, ángulo  

    ↪ inadecuado o distancia excesiva',
    'visible_legible': 'La señal está en buen estado: bien ubicada,  

    ↪ iluminada y legible'
}
defectos.insert(0, diagnosticos_clase.get(clase_pred, ''))

return {
    'clase': clase_pred,

```

```

        'confianza':      confianza,
        'probabilidades': {CLASES[i]: float(probs[i]) * 100 for i in
↪range(NUM_CLASES)},
        'metricas':      {'brillo': brillo, 'contraste': contraste, 'nitidez':
↪nitidez},
        'defectos':      defectos,
        'img_original':   img_pil,
        'img_normalizada': tensor.squeeze(0).cpu()
    }

def mostrar_diagnostico(resultado):
    """Muestra la imagen y el diagnóstico completo de forma visual y textual."""

    # Visualización de imágenes
    img_orig = resultado['img_original']
    img_norm = desnormalizar(resultado['img_normalizada']).permute(1, 2, 0).
↪numpy()
    img_norm = np.clip(img_norm, 0, 1)

    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    axes[0].imshow(img_orig)
    axes[0].set_title('Imagen original', fontweight='bold')
    axes[0].axis('off')
    axes[1].imshow(img_norm)
    axes[1].set_title(f'Normalizada {IMG_SIZE}x{IMG_SIZE}x3', fontweight='bold')
    axes[1].axis('off')
    plt.tight_layout()
    plt.show()

    # Diagnóstico textual
    clase      = resultado['clase']
    confianza  = resultado['confianza']
    es_ok      = clase == 'visible_legible'

    print('=' * 60)
    print(f'  CLASIFICACION: {clase.upper().replace("_", " ")}')
    print(f'  Confianza del modelo: {confianza:.1f}%')
    print('-' * 60)
    print('  Probabilidades por clase:')
    for cls, pct in sorted(resultado['probabilidades'].items(),
                           key=lambda x: x[1], reverse=True):
        barra = '#' * int(pct / 4)
        print(f'    {cls:<28} {pct:5.1f}%  {barra}')
    print('-' * 60)
    print('  Metricas de calidad de la imagen:')
    m = resultado['metricas']

```

```

print(f'    Brillo promedio  : {m["brillo"]:.1f} / 255')
print(f'    Contraste (sigma): {m["contraste"]:.1f}')
print(f'    Nitidez (Lapl.)   : {m["nitidez"]:.1f}')
print('-' * 60)
print(' Diagnostico:')
for d in resultado['defectos']:
    if d:
        marcador = '[OK]' if es_ok else '!'
        print(f'    {marcador} {d}')
print('=' * 60)

print('Funciones de analisis y diagnostico listas.')

```

Funciones de analisis y diagnostico listas.

```

[ ]: # OPCION A: Subir imagen directamente desde tu PC
      # Se abre un selector de archivos; puedes subir una o varias imágenes

from google.colab import files

print('Selecciona una o varias imagenes de senalizacion urbana...')
subidas = files.upload()

for nombre_archivo in subidas.keys():
    ruta = f'/content/{nombre_archivo}'
    print(f'Analizando: {nombre_archivo}')
    resultado = analizar_imagen(ruta)
    mostrar_diagnostico(resultado)

```

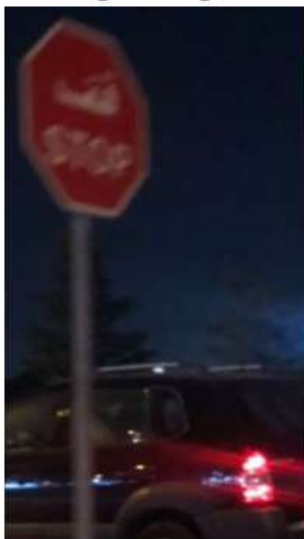
Selecciona una o varias imagenes de senalizacion urbana...

<IPython.core.display.HTML object>

Saving imagenPrueba01.jpg to imagenPrueba01.jpg

Analizando: imagenPrueba01.jpg

Imagen original



Normalizada 512x512x3



```
=====
CLASIFICACION: DETERIORADA
Confianza del modelo: 77.5%
```

```
-----
Probabilidades por clase:
```

deteriorada	77.5%	#####
visible_legible	17.5%	####
poco_visible	5.0%	#

```
-----
Metricas de calidad de la imagen:
```

Brillo promedio	: 32.0 / 255
Contraste (sigma)	: 21.4
Nitidez (Lapl.)	: 10.4

```
-----
Diagnostico:
```

[!] La señal presenta daños físicos: pintura desgastada, oxido, deformación o grafiti

[!] Iluminacion muy baja (brillo=32.0/255): posible mala iluminacion nocturna o sombras

[!] Contraste bajo (sigma=21.4): colores de la señal no se diferencian bien

[!] Imagen muy borrosa (nitidez=10.4): angulo inadecuado, distancia excesiva o movimiento

```
=====
[ ]: # OPCION A: Subir imagen directamente desde tu PC
      # Se abre un selector de archivos; puedes subir una o varias imágenes
```

```

from google.colab import files

print('Selecciona una o varias imagenes de senalizacion urbana...')
subidas = files.upload()

for nombre_archivo in subidas.keys():
    ruta = f'/content/{nombre_archivo}'
    print(f'Analizando: {nombre_archivo}')
    resultado = analizar_imagen(ruta)
    mostrar_diagnostico(resultado)

```

Selecciona una o varias imagenes de senalizacion urbana...

<IPython.core.display.HTML object>

Saving imagenPrueba03.jpg to imagenPrueba03.jpg

Analizando: imagenPrueba03.jpg

Imagen original



Normalizada 512x512x3



=====

CLASIFICACION: POCO VISIBLE

Confianza del modelo: 81.3%

Probabilidades por clase:

poco_visible	81.3%	#####
visible_legible	9.7%	##
deteriorada	9.0%	##

Metricas de calidad de la imagen:

Brillo promedio : 13.2 / 255

Contraste (sigma): 14.5
Nitidez (Lapl.) : 389.2

Diagnostico:

[!] La señal no es legible por mala iluminación, ángulo inadecuado o distancia excesiva

[!] Iluminacion muy baja (brillo=13.2/255): posible mala iluminacion nocturna o sombras

[!] Contraste bajo (sigma=14.5): colores de la señal no se diferencian bien
=====

```
[ ]: # OPCION A: Subir imagen directamente desde tu PC
# Se abre un selector de archivos; puedes subir una o varias imágenes

from google.colab import files

print('Selecciona una o varias imagenes de senalizacion urbana...')
subidas = files.upload()

for nombre_archivo in subidas.keys():
    ruta = f'/content/{nombre_archivo}'
    print(f'Analizando: {nombre_archivo}')
    resultado = analizar_imagen(ruta)
    mostrar_diagnostico(resultado)
```

Selecciona una o varias imagenes de senalizacion urbana...

<IPython.core.display.HTML object>

Saving imagenPrueba04.jpg to imagenPrueba04.jpg

Analizando: imagenPrueba04.jpg

Imagen original



Normalizada 512x512x3



```
=====
CLASIFICACION: DETERIORADA
Confianza del modelo: 73.4%
-----

Probabilidades por clase:
deteriorada          73.4% #####
visible_legible      24.8% #####
poco_visible         1.8%
-----

Metricas de calidad de la imagen:
Brillo promedio   : 37.2 / 255
Contraste (sigma): 25.9
Nitidez (Lapl.)   : 96.3
-----

Diagnostico:
[!] La señal presenta daños físicos: pintura desgastada, oxido, deformación
o grafiti
[!] Iluminacion muy baja (brillo=37.2/255): posible mala iluminacion
nocturna o sombras
[!] Nitidez moderada (nitidez=96.3): posible lejanía o leve desenfoque
=====
```

```
[ ]: # OPCION A: Subir imagen directamente desde tu PC
# Se abre un selector de archivos; puedes subir una o varias imágenes

from google.colab import files

print('Selecciona una o varias imagenes de senalizacion urbana...')
subidas = files.upload()

for nombre_archivo in subidas.keys():
    ruta = f'/content/{nombre_archivo}'
    print(f'Analizando: {nombre_archivo}')
    resultado = analizar_imagen(ruta)
    mostrar_diagnostico(resultado)
```

Selecciona una o varias imagenes de senalizacion urbana...

<IPython.core.display.HTML object>

Saving senalDestruida1.jpg to senalDestruida1.jpg

Analizando: senalDestruida1.jpg



```
=====
CLASIFICACION: DETERIORADA
Confianza del modelo: 97.6%
```

```
-----
Probabilidades por clase:
deteriorada           97.6% #####
visible_legible       2.1%
poco_visible          0.3%
```

```
-----
Metricas de calidad de la imagen:
Brillo promedio   : 149.2 / 255
Contraste (sigma): 52.3
Nitidez (Lapl.)  : 4191.5
```

```
-----
Diagnostico:
[!] La señal presenta daños físicos: pintura desgastada, oxido, deformación
o grafiti
=====
```

```
[ ]: # OPCION A: Subir imagen directamente desde tu PC
# Se abre un selector de archivos; puedes subir una o varias imágenes

from google.colab import files

print('Selecciona una o varias imagenes de senalizacion urbana...')
subidas = files.upload()

for nombre_archivo in subidas.keys():
    ruta = f'/content/{nombre_archivo}'
    print(f'Analizando: {nombre_archivo}')
    resultado = analizar_imagen(ruta)
```



```
mostrar_diagnostico(resultado)
```

Selecciona una o varias imagenes de senalizacion urbana...

<IPython.core.display.HTML object>

Saving senalDestruida2.jpg to senalDestruida2.jpg

Analizando: senalDestruida2.jpg

Imagen original



Normalizada 512x512x3



```
=====
CLASIFICACION: DETERIORADA
Confianza del modelo: 90.9%
```

```
-----
Probabilidades por clase:
```

deteriorada	90.9%	#####
visible_legible	8.4%	##
poco_visible	0.7%	

```
-----
Metricas de calidad de la imagen:
```

Brillo promedio	: 137.4 / 255
Contraste (sigma)	: 46.4
Nitidez (Lapl.)	: 1253.7

```
-----
Diagnostico:
```

[!] La señal presenta daños físicos: pintura desgastada, oxido, deformación o grafiti

```
[ ]: # OPCION A: Subir imagen directamente desde tu PC
# Se abre un selector de archivos; puedes subir una o varias imágenes

from google.colab import files

print('Selecciona una o varias imagenes de senalizacion urbana...')
subidas = files.upload()

for nombre_archivo in subidas.keys():
    ruta = f'/content/{nombre_archivo}'
    print(f'Analizando: {nombre_archivo}')
    resultado = analizar_imagen(ruta)
    mostrar diagnostico(resultado)
```

Selecciona una o varias imagenes de senalizacion urbana...

<IPython.core.display.HTML object>

Saving imagenPrueba05.jpg to imagenPrueba05.jpg

Analizando: imagenPrueba05.jpg



=====

CLASIFICACION: VISIBLE LEGIBLE

Confianza del modelo: 71.9%

Probabilidades por clase:

visible_legible	71.9%	#####
poco_visible	18.3%	####
deteriorada	9.7%	##

Metricas de calidad de la imagen:

Brillo promedio : 175.1 / 255

Contraste (sigma): 55.7
Nitidez (Lapl.) : 873.5

Diagnostico:

[OK] La señal está en buen estado: bien ubicada, iluminada y legible
=====

```
[ ]: # OPCION A: Subir imagen directamente desde tu PC
# Se abre un selector de archivos; puedes subir una o varias imágenes

from google.colab import files

print('Selecciona una o varias imagenes de senalizacion urbana...')
subidas = files.upload()

for nombre_archivo in subidas.keys():
    ruta = f'/content/{nombre_archivo}'
    print(f'Analizando: {nombre_archivo}')
    resultado = analizar_imagen(ruta)
    mostrar_diagnostico(resultado)
```

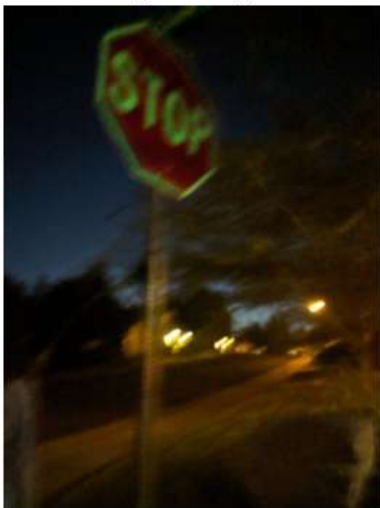
Selecciona una o varias imagenes de senalizacion urbana...

<IPython.core.display.HTML object>

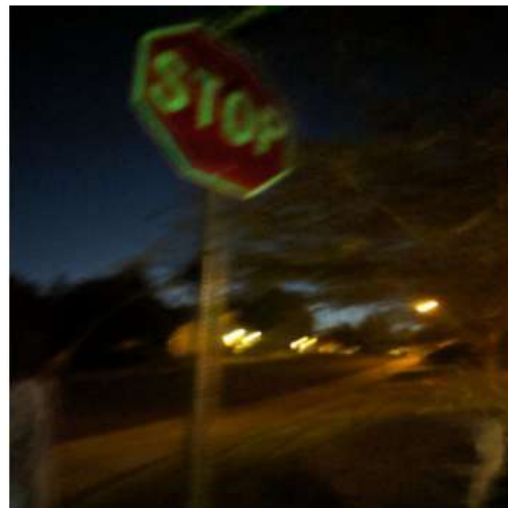
Saving imagenPrueba02.jpg to imagenPrueba02.jpg

Analizando: imagenPrueba02.jpg

Imagen original



Normalizada 512x512x3



=====

CLASIFICACION: VISIBLE LEGIBLE

Confianza del modelo: 56.5%

Probabilidades por clase:

visible_legible	56.5%	#####
deteriorada	30.4%	#####
poco_visible	13.0%	###

Metricas de calidad de la imagen:

Brillo promedio : 28.6 / 255

Contraste (sigma): 23.2

Nitidez (Lapl.) : 14.5

Diagnostico:

[OK] La señal está en buen estado: bien ubicada, iluminada y legible

[OK] Iluminacion muy baja (brillo=28.6/255): posible mala iluminacion nocturna o sombras

[OK] Contraste bajo (sigma=23.2): colores de la señal no se diferencian bien

[OK] Imagen muy borrosa (nitidez=14.5): angulo inadecuado, distancia excesiva o movimiento

=====

1.12 10. Análisis crítico y conclusiones

1.12.1 Respuesta a la pregunta de investigación

Sí, una CNN puede clasificar el estado visual de la señalización urbana y apoyar una auditoría visual preliminar. El modelo aprende a distinguir 3 tipos de señalización a partir de imágenes RGB de 512×512 píxeles.

1.12.2 Resumen de cada clase

Clase	Imágenes	Características aprendidas	Riesgo de confusión
visible_legible	80	Colores vivos, bordes nítidos, fondo limpio	Baja
deteriorada	80	Manchas, texturas rugosas, colores apagados	Media
poco_visible	80	Brillo bajo/alto, bajo contraste, desenfoque	Media

1.12.3 Por qué Cross Validation con k=5 mejora la clasificación

Con datasets de tamaño moderado (80 imgs/clase), una sola partición aleatoria puede producir splits desfavorables. K-Fold CV garantiza que **cada imagen aparece exactamente una vez en validación**, promediando el rendimiento sobre 5 particiones distintas y reduciendo la varianza del estimador de generalización.

1.12.4 Limitaciones

1. **Dataset moderado** (80 imágenes/clase): mitigado con AutoAugment y CV.

2. **Clases similares:** `deteriorada` y `poco_visible` comparten características visuales en casos límite; se recomienda revisar manualmente los errores.
3. **Resolución 512×512 :** mayor detalle pero mayor costo computacional.

1.12.5 Posibles mejoras futuras

- Aplicar **Transfer Learning** con EfficientNet-B0 o ResNet18 preentrenados.
- Usar **RandAugment** como alternativa más controlable a TrivialAugmentWide.
- Incorporar **detección de objetos** (YOLOv8) para localizar la señal antes de clasificar.
- Exportar a **TFLite** / **ONNX** para uso en dispositivos móviles en auditorías de campo.
- Aumentar a **k=10** si el dataset crece a 200+ imágenes por clase.